

Module 6 Notes: Searching and Improving Search in a Linked List

1. Current progress analysis

You are ready for Module 6 because you already understand the ideas that this module depends on.

From Module 1

You learned why linked lists exist.

Arrays are fast when we already know the index, but arrays have a fixed size after creation. Linked lists are more flexible because they can grow and shrink one node at a time.

Basic tradeoff:

Structure	Strength	Weakness
Array	Fast direct access by index	Fixed size, shifting needed during insertion/deletion
Linked list	Flexible size, easier pointer-based insertion/deletion	No direct jumping to an index

This matters for Module 6 because searching depends heavily on whether we can jump directly to a position.

Arrays can jump to an index.

Linked lists cannot.

From Module 2

You learned what a node is.

A singly linked-list node has:

[data | next]

Meaning:

Part	Meaning
data	The value stored in the node
next	Address of the next node

C node definition:

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

You also learned this important pointer movement:

```
p = p->next;
```

This moves pointer `p` forward by one node.

From Module 3

You learned traversal.

Traversal means visiting nodes one by one from the first node until `NULL`.

Example:

```
first -> 8 -> 3 -> 7 -> 12 -> NULL
```

Basic traversal pattern:

```
while (p != NULL) {  
    // use p->data  
    p = p->next;  
}
```

Important rule:

Do not move first directly during traversal.
Use a temporary pointer like p.

This rule is important in Module 6 because search also moves through the list.

From Module 4

You learned recursive traversal.

A recursive linked-list function usually has a base case like:

```
if (p == NULL) {  
    return some_value;  
}
```

`p == NULL` means the list has ended.

Module 6 uses this idea for recursive search.

From Module 5

You learned that many linked-list operations are traversal plus some work at each node.

Examples:

Operation	Work done at each node
Count	Add 1
Sum	Add <code>p->data</code>
Maximum	Compare <code>p->data</code> with current maximum
Minimum	Compare <code>p->data</code> with current minimum

Module 6 follows the same pattern.

For search, the work at each node is:

Check whether `p->data` is equal to the key.

2. Big idea of Module 6

Module 6 teaches how to search for a value in a singly linked list.

Given this linked list:

```
8 -> 3 -> 7 -> 12 -> 9 -> NULL
```

Suppose we want to search for:

```
12
```

A linked list cannot jump directly to the node containing `12`.

It must start from the first node and move one node at a time.

Search process:

```
Check 8  
Check 3  
Check 7  
Check 12  
Found
```

This is called **linear search**.

Simple definition

Linear search means:

```
Start from the first node and check each node one by one until the key is found  
or the list ends.
```

3. What is a key?

In search operations, the value we are looking for is usually called the **key**.

Example:

```
List: 8 -> 3 -> 7 -> 12 -> 9 -> NULL  
Key: 12
```

Here, 12 is the key.

The search function checks each node and asks:

```
Is this node's data equal to the key?
```

In C:

```
if (key == p->data)
```

4. Why linked lists use linear search

A singly linked list only knows the address of the next node.

Example:

```
8 -> 3 -> 7 -> 12 -> 9 -> NULL
```

The node containing 8 knows where 3 is.

The node containing 3 knows where 7 is.

The node containing 7 knows where 12 is.

But the list does not have direct index access like an array.

In an array, we can write:

```
A[3]
```

This directly accesses index 3.

In a linked list, we cannot write:

```
list[3]
```

At least not in a normal C linked-list implementation.

To reach the fourth node, we must move like this:

```
first -> second -> third -> fourth
```

So the natural search method for linked lists is linear search.

5. Why binary search is not good for linked lists

Binary search works by repeatedly checking the middle element.

For a sorted array:

```
[3, 7, 8, 12, 20, 25, 30]
```

Binary search can directly check the middle element using an index:

```
A[mid]
```

Array access by index is fast:

```
O(1)
```

That is why binary search works well on sorted arrays.

Its time complexity on a sorted array is:

```
O(log n)
```

But in a linked list, getting the middle node is not direct.

Example linked list:

```
3 -> 7 -> 8 -> 12 -> 20 -> 25 -> 30 -> NULL
```

To reach the middle node, we must start from the first node and move step by step.

So even if the linked list is sorted, binary search loses its main advantage.

Important conclusion

```
Binary search is efficient for sorted arrays.  
Binary search is inefficient for linked lists.
```

Why?

```
Because linked lists cannot jump to the middle node in  $O(1)$  time.
```

6. Iterative linear search

Iterative search uses a loop.

The idea is:

```
Start p at the first node.  
While p is not NULL:  
    Check p->data.  
    If it matches the key, return p.  
    Otherwise move p to the next node.  
If the loop ends, return NULL.
```

7. What should search return?

The search function usually returns the **address of the found node**, not just the value.

If the key is found:

```
Return pointer to that node.
```

If the key is not found:

```
Return NULL.
```

Why return the node address?

Because returning the node allows the program to use the found node later.

For example, we may want to:

- Print the node's data
- Modify the node
- Move the node
- Use the node's address in another operation

So the return type is:

```
struct Node *
```

That means:

```
This function returns a pointer to a Node.
```

8. Iterative search code

```
struct Node *Search(struct Node *p, int key) {  
    while (p != NULL) {  
        if (key == p->data) {  
            return p;  
        }  
        p = p->next;  
    }  
    return NULL;  
}
```

9. Explanation of iterative search code

Function header

```
struct Node *Search(struct Node *p, int key)
```

This means:

Search receives a pointer p and an integer key.
It returns a pointer to a Node.

Usually, we call it like this:

```
struct Node *result;  
result = Search(first, 12);
```

Here, `first` points to the first node of the list.

Loop condition

```
while (p != NULL)
```

This means:

Keep searching while p points to a real node.

When `p` becomes `NULL`, the list has ended.

Comparison

```
if (key == p->data)
```

This checks whether the current node stores the key.

Example:

```
key = 12  
p->data = 12
```

Then the key is found.

Return found node

```
return p;
```

This returns the address of the node where the key was found.

Move forward

```
p = p->next;
```

This moves `p` to the next node.

Return NULL

```
return NULL;
```

This line runs only if the whole list was searched and the key was not found.

10. Dry run: successful iterative search

List:

```
8 -> 3 -> 7 -> 12 -> 9 -> NULL
```

Search key:

```
12
```

Function call:

```
Search(first, 12);
```

Trace:

Step	p points to	Comparison	Result
1	8	12 == 8	False. Move to next node.
2	3	12 == 3	False. Move to next node.
3	7	12 == 7	False. Move to next node.
4	12	12 == 12	True. Return this node.

Final result:

```
Search returns the address of the node containing 12.
```

11. Dry run: unsuccessful iterative search

List:

```
8 -> 3 -> 7 -> 12 -> 9 -> NULL
```

Search key:

```
25
```

Function call:

```
Search(first, 25);
```

Trace:

Step	p points to	Comparison	Result
1	8	25 == 8	False. Move to next node.
2	3	25 == 3	False. Move to next node.
3	7	25 == 7	False. Move to next node.

Step	p points to	Comparison	Result
4	12	25 == 12	False. Move to next node.
5	9	25 == 9	False. Move to next node.
6	NULL	Stop	Return NULL .

Final result:

Search returns NULL because 25 is not in the list.

12. Searching an empty list

An empty list means:

`first == NULL`

If we call:

`Search(first, 12);`

Then `p` is already `NULL`.

The loop does not run.

The function returns:

`NULL`

This is correct because an empty list has no nodes.

13. Recursive linear search

Recursive search performs the same search, but uses function calls instead of a loop.

The idea is:

If p is NULL, the key is not found.
If p->data equals key, the key is found.
Otherwise, search the rest of the list.

The rest of the list starts at:

p->next

14. Recursive search base cases

Recursive search has two important base cases.

Base case 1: list ended

```
if (p == NULL) {  
    return NULL;  
}
```

Meaning:

There are no more nodes to check.
The key was not found.

Base case 2: key found

```
if (key == p->data) {  
    return p;  
}
```

Meaning:

The current node contains the key.
Return this node.

15. Recursive search code

```
struct Node *RSearch(struct Node *p, int key) {  
    if (p == NULL) {  
        return NULL;  
    }  
  
    if (key == p->data) {  
        return p;  
    }  
  
    return RSearch(p->next, key);  
}
```

16. Explanation of recursive search code

Check for end of list

```
if (p == NULL) {  
    return NULL;  
}
```

If `p` is `NULL`, there is no node to check.

So the key is not found.

Check current node

```
if (key == p->data) {  
    return p;  
}
```

If the current node stores the key, return the current node.

Search the rest

```
return RSearch(p->next, key);
```

If the current node does not contain the key, search from the next node.

17. Dry run: recursive search, key found

List:

```
8 -> 3 -> 7 -> 12 -> 9 -> NULL
```

Search key:

```
12
```

Function call:

```
RSearch(first, 12);
```

Trace:

```
RSearch(8, 12)
8 is not 12, so search next.

RSearch(3, 12)
3 is not 12, so search next.

RSearch(7, 12)
7 is not 12, so search next.

RSearch(12, 12)
12 is 12, so return this node.
```

Final result:

```
The function returns the address of the node containing 12.
```

18. Dry run: recursive search, key not found

List:

8 -> 3 -> 7 -> 12 -> 9 -> NULL

Search key:

25

Trace:

```
RSearch(8, 25)
RSearch(3, 25)
RSearch(7, 25)
RSearch(12, 25)
RSearch(9, 25)
RSearch(NULL, 25)
```

At `NULL`, the function returns:

NULL

Final result:

The key was not found.

19. Iterative search vs recursive search

Both versions do the same logical work.

Feature	Iterative search	Recursive search
Main tool	<code>while</code> loop	Function calls
End condition	<code>p == NULL</code>	<code>p == NULL</code>
Found condition	<code>key == p->data</code>	<code>key == p->data</code>
Time complexity	<code>O(n)</code> worst case	<code>O(n)</code> worst case
Extra space	<code>O(1)</code>	<code>O(n)</code> due to system stack

Important point

Recursive search is clean, but it uses extra stack memory.

For very large lists, iterative search is usually safer.

20. Improving search

Normal linear search always starts from the first node.

Example:

```
8 -> 3 -> 7 -> 12 -> 9 -> NULL
```

Searching for 12 requires four comparisons:

```
8, 3, 7, 12
```

If we search for 12 only once, this is fine.

But if we search for 12 many times, repeating four comparisons every time is wasteful.

So we can improve future searches by moving the found node to the front.

This method is called:

```
move-to-head
```

21. What is move-to-head?

Move-to-head means:

```
When a key is found, move that node to the beginning of the list.
```

Before searching:

```
8 -> 3 -> 7 -> 12 -> 9 -> NULL
```

Search for:

```
12
```

After move-to-head:

```
12 -> 8 -> 3 -> 7 -> 9 -> NULL
```

Now, if we search for `12` again, it is found immediately at the first node.

22. Why move-to-head can help

Move-to-head helps when some values are searched repeatedly.

Example:

```
Initial list:  
8 -> 3 -> 7 -> 12 -> 9 -> NULL
```

First search for `12`:

```
Check 8  
Check 3  
Check 7  
Check 12  
Found
```

Then move `12` to the front:

```
12 -> 8 -> 3 -> 7 -> 9 -> NULL
```

Second search for `12`:

Check 12
Found

So the repeated search becomes faster.

23. Important warning about move-to-head

Move-to-head improves future searches for the same key, but it does not make the first search faster.

The first search still has to find the key.

So if the key is near the end, the first search is still:

$O(n)$

But after the node is moved to the head, the next search for the same key can be:

$O(1)$

24. Why move-to-head needs two pointers

To move a node to the head, we must unlink it from its current position.

For this list:

8 -> 3 -> 7 -> 12 -> 9 -> NULL

Search for 12.

When p reaches 12, we also need to know the node before 12.

The node before 12 is 7.

So we use two pointers:

Pointer	Role
p	Current search pointer

Pointer	Role
<code>q</code>	Trailing pointer, follows one node behind <code>p</code>

At the moment `12` is found:

```
8 -> 3 -> 7 -> 12 -> 9 -> NULL
      q      p
```

`p` points to the found node.

`q` points to the previous node.

25. Why `q` is necessary

To remove `12` from its current position, we must make `7` point to `9`.

That means:

```
q->next = p->next;
```

In the example:

```
q is 7
p is 12
p->next is 9
```

So:

```
q->next = p->next;
```

means:

```
7 now points to 9.
```

Without `q`, we would not know which node should skip over `p`.

26. Move-to-head pointer changes

Before move-to-head:

```
first
|
v
8 -> 3 -> 7 -> 12 -> 9 -> NULL
      q      p
```

Goal:

```
12 -> 8 -> 3 -> 7 -> 9 -> NULL
```

Three pointer changes are needed.

Change 1: unlink the found node

```
q->next = p->next;
```

Before:

```
7 -> 12 -> 9
```

After:

```
7 -> 9
```

Now the list without **12** is:

```
8 -> 3 -> 7 -> 9 -> NULL
```

The node **12** is not deleted. It is just separated from its old position.

Change 2: connect found node to old first

```
p->next = first;
```

This makes 12 point to the old first node, which is 8.

Now:

```
12 -> 8 -> 3 -> 7 -> 9 -> NULL
```

Change 3: update first

```
first = p;
```

This makes 12 the new first node.

Final list:

```
12 -> 8 -> 3 -> 7 -> 9 -> NULL
```

27. Move-to-head search code

This version assumes first is a global pointer.

```
struct Node *SearchImprove(struct Node *p, int key) {
    struct Node *q = NULL;

    while (p != NULL) {
        if (key == p->data) {
            if (q != NULL) {
                q->next = p->next;
                p->next = first;
                first = p;
            }
            return p;
        }
        q = p;
        p = p->next;
    }
    return NULL;
}
```

```
        p = p->next;
    }

    return NULL;
}
```

28. Explanation of move-to-head code

Create trailing pointer

```
struct Node *q = NULL;
```

At the beginning, `p` points to the first node.

There is no previous node before the first node.

So `q` starts as `NULL`.

Search loop

```
while (p != NULL)
```

Keep searching until the list ends.

Found condition

```
if (key == p->data)
```

If the current node contains the key, the node has been found.

Check whether found node is already first

```
if (q != NULL)
```

If `q == NULL`, then the found node is already the first node.

Example:

```
8 -> 3 -> 7 -> NULL
p
```

If we search for 8, p is at the first node and q is NULL.

No move is needed.

So we only perform move-to-head when:

```
q != NULL
```

That means the found node is not already first.

Move-to-head lines

```
q->next = p->next;
p->next = first;
first = p;
```

Meaning:

Line	Meaning
q->next = p->next;	Previous node skips the found node
p->next = first;	Found node points to old first node
first = p;	Found node becomes the new first node

Move q and p forward

```
q = p;
p = p->next;
```

These lines run when the current node is not the key.

They move both pointers forward while keeping q one step behind p.

Example:

Before:
q = NULL, p = 8

After:
q = 8, p = 3

Then:

q = 3, p = 7

Then:

q = 7, p = 12

29. Dry run: move-to-head search

List:

8 -> 3 -> 7 -> 12 -> 9 -> NULL

Search key:

12

Initial state:

p = 8
q = NULL
first = 8

Trace:

Step	q points to	p points to	Check
1	NULL	8	12 == 8 is false
2	8	3	12 == 3 is false
3	3	7	12 == 7 is false

Step	q points to	p points to	Check
4	7	12	12 == 12 is true

At step 4:

```
q points to 7
p points to 12
```

Now perform the three pointer changes.

Pointer change 1

```
q->next = p->next;
```

Meaning:

```
7 now points to 9.
```

List becomes:

```
8 -> 3 -> 7 -> 9 -> NULL
```

Node 12 is separated.

Pointer change 2

```
p->next = first;
```

Meaning:

```
12 now points to 8.
```

Pointer change 3

```
first = p;
```

Meaning:

```
first now points to 12.
```

Final list:

```
12 -> 8 -> 3 -> 7 -> 9 -> NULL
```

30. Case: key already at the head

List:

```
8 -> 3 -> 7 -> NULL
```

Search key:

```
8
```

Initial:

```
p = 8  
q = NULL
```

The key is found immediately.

Since `q == NULL`, the node is already first.

So this block does not run:

```
if (q != NULL) {  
    q->next = p->next;  
    p->next = first;  
}
```

```
    first = p;  
}
```

The list remains:

```
8 -> 3 -> 7 -> NULL
```

This is correct.

31. Case: key at the tail

List:

```
8 -> 3 -> 7 -> 12 -> NULL
```

Search key:

```
12
```

At the moment the key is found:

```
8 -> 3 -> 7 -> 12 -> NULL  
          q     p
```

Now:

```
q->next = p->next;
```

Since `p->next` is `NULL`, this means:

```
7 -> NULL
```

Then:

```
p->next = first;  
first = p;
```

Final list:

```
12 -> 8 -> 3 -> 7 -> NULL
```

So move-to-head works even when the key is at the tail.

32. Case: key not found with move-to-head

List:

```
8 -> 3 -> 7 -> NULL
```

Search key:

```
50
```

The search checks all nodes:

```
8, 3, 7
```

Then `p` becomes `NULL`.

The function returns:

```
NULL
```

No links are changed.

The list remains:

```
8 -> 3 -> 7 -> NULL
```

33. Move-to-head does not delete the node

A common confusion is thinking this line deletes the found node:

```
q->next = p->next;
```

It does not delete the node.

It only removes the node from its old position in the chain.

The node still exists because `p` still points to it.

Then the node is attached to the front:

```
p->next = first;  
first = p;
```

So move-to-head is a re-linking operation, not a deletion operation.

34. Transposition idea

Another possible search improvement is called **transposition**.

Transposition means:

```
When a node is found, swap it with the previous node.
```

Example:

Before:

```
8 -> 3 -> 7 -> 12 -> 9 -> NULL
```

If `12` is found, transposition could move it one step forward:

```
8 -> 3 -> 12 -> 7 -> 9 -> NULL
```

But the source material discourages swapping raw data in linked lists.

Why?

Because linked-list nodes may contain large records, not just small integers.

Swapping large data can be expensive or unsafe.

Move-to-head is the main improvement emphasized in this module.

35. Complexity of search operations

Let n be the number of nodes in the list.

Best case: key at the first node

Example:

```
12 -> 8 -> 3 -> 7 -> NULL
```

Search key:

```
12
```

Only one comparison is needed.

Time:

```
 $O(1)$ 
```

Worst case: key at the last node

Example:

```
8 -> 3 -> 7 -> 12 -> NULL
```

Search key:

```
12
```

The search may check every node.

Time:

$O(n)$

Not-found case

Example:

8 -> 3 -> 7 -> 12 -> NULL

Search key:

50

The search must check every node before deciding the key is not present.

Time:

$O(n)$

36. Complexity table

Operation	Time complexity	Extra space	Reason
Iterative search, best case	$O(1)$	$O(1)$	Key is at first node
Iterative search, worst case	$O(n)$	$O(1)$	Key is at last node or near end
Search key not found	$O(n)$	$O(1)$	Must check all nodes
Recursive search	$O(n)$	$O(n)$	Function calls use stack space
First move-to-head search	$O(n)$	$O(1)$	Must find the key first
Repeated search after move-to-head	$O(1)$	$O(1)$	Key is now at the first node

37. Full C program for Module 6

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *first = NULL;

void create(int A[], int n) {
    if (n <= 0) {
        first = NULL;
        return;
    }

    struct Node *t;
    struct Node *last;

    first = (struct Node *)malloc(sizeof(struct Node));
    first->data = A[0];
    first->next = NULL;
    last = first;

    for (int i = 1; i < n; i++) {
        t = (struct Node *)malloc(sizeof(struct Node));
        t->data = A[i];
        t->next = NULL;
        last->next = t;
        last = t;
    }
}

void display(struct Node *p) {
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
}

struct Node *Search(struct Node *p, int key) {
    while (p != NULL) {
        if (key == p->data) {
            return p;
        }
    }
}
```

```

        }
        p = p->next;
    }
    return NULL;
}

struct Node *RSearch(struct Node *p, int key) {
    if (p == NULL) {
        return NULL;
    }

    if (key == p->data) {
        return p;
    }

    return RSearch(p->next, key);
}

struct Node *SearchImprove(struct Node *p, int key) {
    struct Node *q = NULL;

    while (p != NULL) {
        if (key == p->data) {
            if (q != NULL) {
                q->next = p->next;
                p->next = first;
                first = p;
            }
            return p;
        }

        q = p;
        p = p->next;
    }

    return NULL;
}

int main() {
    int A[] = {8, 3, 7, 12, 9};
    struct Node *result;

    create(A, 5);

    printf("Original list: ");
    display(first);

    result = Search(first, 12);
}

```

```
    if (result != NULL) {
        printf("\nIterative search: key found: %d", result->data);
    } else {
        printf("\nIterative search: key not found");
    }

    result = RSearch(first, 25);
    if (result != NULL) {
        printf("\nRecursive search: key found: %d", result->data);
    } else {
        printf("\nRecursive search: key not found");
    }

    result = SearchImprove(first, 12);
    if (result != NULL) {
        printf("\nMove-to-head search: key found: %d", result->data);
    } else {
        printf("\nMove-to-head search: key not found");
    }

    printf("\nList after move-to-head search: ");
    display(first);

    return 0;
}
```

38. Expected output idea

```
Original list: 8 3 7 12 9
Iterative search: key found: 12
Recursive search: key not found
Move-to-head search: key found: 12
List after move-to-head search: 12 8 3 7 9
```

The exact output format can change, but the important final list is:

```
12 8 3 7 9
```

39. Common beginner mistakes

Mistake 1: Thinking binary search is good for linked lists

Wrong idea:

```
The list is sorted, so binary search must be efficient.
```

Correct idea:

```
Binary search needs fast access to the middle.  
Linked lists cannot jump to the middle in  $O(1)$ .
```

Mistake 2: Returning `p->data` instead of `p`

Wrong:

```
return p->data;
```

Correct:

```
return p;
```

Why?

Because the function is supposed to return the address of the found node.

The return type is:

```
struct Node *
```

So the function must return a node pointer, not an integer.

Mistake 3: Forgetting to move `p`

Wrong:

```
while (p != NULL) {  
    if (key == p->data) {  
        return p;  
    }  
}
```

Problem:

p never moves to the next node.
The loop can run forever if the first node is not the key.

Correct:

```
while (p != NULL) {  
    if (key == p->data) {  
        return p;  
    }  
    p = p->next;  
}
```

Mistake 4: Accessing `p->data` when `p` is `NULL`

Wrong:

```
while (p->data != key) {  
    p = p->next;  
}
```

Problem:

If `p` becomes `NULL`, then `p->data` is invalid.

Correct:

```
while (p != NULL) {  
    if (key == p->data) {  
        return p;  
    }  
}
```

```
p = p->next;  
}
```

Always check that `p` is not `NULL` before using `p->data`.

Mistake 5: Forgetting `q` in move-to-head

Wrong idea:

Only `p` is needed to move the found node.

Correct idea:

`q` is needed because `q` points to the node before `p`.

Without `q`, we cannot safely skip over the found node.

Important line:

```
q->next = p->next;
```

Mistake 6: Not checking whether the found node is already first

Dangerous:

```
q->next = p->next;
```

If `p` is the first node, then `q` is `NULL`.

So `q->next` is invalid.

Correct:

```
if (q != NULL) {  
    q->next = p->next;  
    p->next = first;  
}
```

```
    first = p;  
}
```

Mistake 7: Thinking move-to-head reduces every search to $O(1)$

Wrong idea:

Move-to-head makes search always $O(1)$.

Correct idea:

The first search can still be $O(n)$.
Repeated search for the same moved key can be $O(1)$.

40. Key patterns to memorize

Iterative search pattern

```
struct Node *Search(struct Node *p, int key) {  
    while (p != NULL) {  
        if (key == p->data) {  
            return p;  
        }  
        p = p->next;  
    }  
    return NULL;  
}
```

Recursive search pattern

```
struct Node *RSearch(struct Node *p, int key) {  
    if (p == NULL) {  
        return NULL;  
    }  
  
    if (key == p->data) {  
        return p;  
    }  
}
```

```
    return RSearch(p->next, key);  
}
```

Move-to-head pattern

```
q->next = p->next;  
p->next = first;  
first = p;
```

These three lines mean:

1. Remove p from its old position.
2. Make p point to the old first node.
3. Make first point to p.

41. Practice dry run 1

Given list:

```
10 -> 20 -> 30 -> 40 -> NULL
```

Search key:

```
30
```

Search path:

```
Check 10  
Check 20  
Check 30  
Found
```

Number of comparisons:

```
3
```

42. Practice dry run 2

Given list:

10 -> 20 -> 30 -> 40 -> NULL

Search key:

50

Search path:

Check 10
Check 20
Check 30
Check 40
Reach NULL
Not found

Number of comparisons:

4

Return value:

NULL

43. Practice dry run 3: move-to-head

Given list:

10 -> 20 -> 30 -> 40 -> NULL

Search key:

30

At the moment 30 is found:

```
10 -> 20 -> 30 -> 40 -> NULL
      q      p
```

Pointer changes:

```
q->next = p->next;
```

Now:

```
10 -> 20 -> 40 -> NULL
```

Then:

```
p->next = first;
```

Now:

```
30 -> 10 -> 20 -> 40 -> NULL
```

Then:

```
first = p;
```

Final list:

```
30 -> 10 -> 20 -> 40 -> NULL
```

44. Practice questions

Question 1

Why is binary search inefficient on a linked list?

Answer:

Because binary search needs fast access to the middle element, but a linked list must move node by node to reach the middle.

Question 2

What does iterative search return if the key is found?

Answer:

It returns the address of the node containing the key.

Question 3

What does iterative search return if the key is not found?

Answer:

It returns `NULL`.

Question 4

What are the two base cases in recursive search?

Answer:

```
p == NULL      -> key not found
key == p->data  -> key found
```

Question 5

In move-to-head search, what is the role of `p`?

Answer:

`p` points to the current node being checked. When the key is found, `p` points to the found node.

Question 6

In move-to-head search, what is the role of `q`?

Answer:

`q` follows one node behind `p`. It points to the previous node, which is needed to unlink the found node from its old position.

Question 7

What are the three main pointer changes in move-to-head?

Answer:

```
q->next = p->next;  
p->next = first;  
first = p;
```

Question 8

Why do we check `q != NULL` before moving a found node to the head?

Answer:

Because if `q == NULL`, the found node is already the first node. Trying to use `q->next` would be invalid.

Question 9

What is the worst-case time complexity of normal linear search?

Answer:

```
O(n)
```

Because the search may need to check every node.

Question 10

When can move-to-head make a future search $O(1)$?

Answer:

After a successful search moves the key to the first node, a repeated search for the same key can be $O(1)$.

45. Final Module 6 summary

Module 6 teaches searching in a singly linked list.

The main search method is:

linear search

Linear search checks nodes one by one:

Start at first.
Check current node.
Move to next node.
Stop when key is found or p becomes NULL.

Iterative search uses a loop:

```
while (p != NULL) {  
    if (key == p->data) {  
        return p;  
    }  
    p = p->next;  
}  
return NULL;
```

Recursive search uses function calls:

```
if (p == NULL) return NULL;  
if (key == p->data) return p;  
return RSearch(p->next, key);
```

Binary search is not suitable for linked lists because linked lists cannot jump directly to the middle node.

Move-to-head improves repeated successful searches.

Before:

```
8 -> 3 -> 7 -> 12 -> 9 -> NULL
```

After searching for 12:

```
12 -> 8 -> 3 -> 7 -> 9 -> NULL
```

Important pointer roles:

Pointer	Meaning
first	Head pointer, points to the first node
p	Current search pointer
q	Previous/trailing pointer

Most important move-to-head lines:

```
q->next = p->next;  
p->next = first;  
first = p;
```

Most important complexity facts:

Case	Complexity
Key at first node	$O(1)$
Key at last node	$O(n)$
Key not found	$O(n)$
Recursive search space	$O(n)$
Move-to-head repeated same key	$O(1)$

The key lesson of Module 6:

Searching in a linked list is traversal plus comparison.
Move-to-head does not make the first search faster, but it can make repeated searches for the same value faster.